

```
In [1]: import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import load_digits
```

```
In [2]: digits = load_digits()

X = digits.data
y = digits.target

print(f"X Shape is {X.shape}")
print(f"y Shape is {y.shape}")
```

```
X Shape is (1797, 64)
y Shape is (1797,)
```

```
In [3]: def train_test_split(X,y,test_size=0.2,random_state=None):
    if random_state is not None:
        np.random.seed(random_state)

    indices = np.arange(X.shape[0])
    np.random.shuffle(indices)

    test_count = int(X.shape[0]*test_size)

    test_indx = indices[:test_count]
    train_indx = indices[test_count:]

    return X[train_indx] , X[test_indx] , y[train_indx] , y[test_indx]
```

```
In [5]: def normalized_data(X):
    X_normalized = X / X.max()
    return X_normalized

X = normalized_data(X)

print(f'Min X {X.min()}')
print(f'Max X {X.max()}')
```

```
Min X 0.0
Max X 1.0
```

```
In [6]: X_train , X_test , y_train , y_test = train_test_split(X,y,test_size=0.2,random

print(f"Training set is : {X_train.shape}")
print(f"Testing set is : {X_test.shape}")
print(f"label training is : {y_train.shape}")
print(f"y_test (target) is : {y_test.shape}")
```

```
Training set is : (1438, 64)
Testing set is : (359, 64)
label training is : (1438,)
y_test (target) is : (359,)
```

```
In [7]: def relu(Z):
    return np.maximum(0,Z)
```

```
def softmax(Z):
    exp_z = np.exp(Z-np.max(Z,axis=1,keepdims=True))
    return exp_z / np.sum(exp_z,axis=1 , keepdims = True)
```

```
In [8]: def cross_entropy(y_true,y_pred):
        epsilon = 1e-8
        y_pred = np.clip(y_pred,epsilon,1-epsilon)
        loss = -np.mean(np.sum(y_true*np.log(y_pred),axis=1))
        return loss
```

```
In [9]: def onehot_encoding(y,num_classes=10):
        one_hot = np.zeros((len(y),num_classes))
        one_hot[np.arange(len(y)),y] = 1
        return one_hot
```

```
In [10]: def train(X_train,y_train,epochs=1000,learning_rate=0.1):
        input_size = X_train.shape[1]
        hidden_size = 128
        output_size = 10
        m = X_train.shape[0]

        w1 = np.random.randn(input_size,hidden_size)*0.01
        b1 = np.zeros((1,hidden_size))

        w2 = np.random.randn(hidden_size,output_size)*0.01
        b2 = np.zeros((1,output_size))

        y_train_encoded = onehot_encoding(y_train)
        for epoch in range(epochs):
            neto1 = X_train@w1+b1
            A1 = relu(neto1)

            neto2 = A1@w2+b2
            A2 = softmax(neto2)

            loss = cross_entropy(y_train_encoded,A2)
            # print(f"Epoch:{epoch} Loss : {loss}")

            dZ2 = A2 - y_train_encoded
            dW2 = (A1.T @ dZ2) / len(X_train)

            db2 = np.sum(dZ2, axis=0, keepdims=True) / len(X_train)

            w2 -= learning_rate * dW2
            b2 -= learning_rate * db2

            dA1 = dZ2 @ w2.T
            dZ1 = dA1 * (neto1>0)
            dW1 = (X_train.T @ dZ1) / m
            db1 = np.sum(dZ1,axis=0,keepdims=True) / len(X_train)

            w1 -= learning_rate * dW1
```

```
        b1 -= learning_rate * db1

    if epoch % 10 == 0:
        print(f"Epoch: {epoch}, Loss: {loss:.4f}")
    return w1, b1, w2, b2

# train(X_train,y_train)
```

```
In [11]: w1, b1, w2, b2 = train(
        X_train,
        y_train,
        epochs=1000,
        learning_rate=0.1
    )
```

Epoch: 0, Loss: 2.3026
Epoch: 10, Loss: 2.2990
Epoch: 20, Loss: 2.2941
Epoch: 30, Loss: 2.2866
Epoch: 40, Loss: 2.2746
Epoch: 50, Loss: 2.2549
Epoch: 60, Loss: 2.2222
Epoch: 70, Loss: 2.1694
Epoch: 80, Loss: 2.0878
Epoch: 90, Loss: 1.9702
Epoch: 100, Loss: 1.8180
Epoch: 110, Loss: 1.6435
Epoch: 120, Loss: 1.4649
Epoch: 130, Loss: 1.2967
Epoch: 140, Loss: 1.1458
Epoch: 150, Loss: 1.0144
Epoch: 160, Loss: 0.9019
Epoch: 170, Loss: 0.8066
Epoch: 180, Loss: 0.7262
Epoch: 190, Loss: 0.6586
Epoch: 200, Loss: 0.6015
Epoch: 210, Loss: 0.5530
Epoch: 220, Loss: 0.5115
Epoch: 230, Loss: 0.4757
Epoch: 240, Loss: 0.4444
Epoch: 250, Loss: 0.4170
Epoch: 260, Loss: 0.3927
Epoch: 270, Loss: 0.3711
Epoch: 280, Loss: 0.3517
Epoch: 290, Loss: 0.3343
Epoch: 300, Loss: 0.3186
Epoch: 310, Loss: 0.3044
Epoch: 320, Loss: 0.2914
Epoch: 330, Loss: 0.2796
Epoch: 340, Loss: 0.2688
Epoch: 350, Loss: 0.2588
Epoch: 360, Loss: 0.2497
Epoch: 370, Loss: 0.2412
Epoch: 380, Loss: 0.2334
Epoch: 390, Loss: 0.2262
Epoch: 400, Loss: 0.2194
Epoch: 410, Loss: 0.2131
Epoch: 420, Loss: 0.2072
Epoch: 430, Loss: 0.2017
Epoch: 440, Loss: 0.1965
Epoch: 450, Loss: 0.1917
Epoch: 460, Loss: 0.1871
Epoch: 470, Loss: 0.1827
Epoch: 480, Loss: 0.1786
Epoch: 490, Loss: 0.1748
Epoch: 500, Loss: 0.1711
Epoch: 510, Loss: 0.1676
Epoch: 520, Loss: 0.1642
Epoch: 530, Loss: 0.1611
Epoch: 540, Loss: 0.1580
Epoch: 550, Loss: 0.1551
Epoch: 560, Loss: 0.1524
Epoch: 570, Loss: 0.1497
Epoch: 580, Loss: 0.1472
Epoch: 590, Loss: 0.1448

```
Epoch: 600, Loss: 0.1424
Epoch: 610, Loss: 0.1402
Epoch: 620, Loss: 0.1380
Epoch: 630, Loss: 0.1360
Epoch: 640, Loss: 0.1340
Epoch: 650, Loss: 0.1320
Epoch: 660, Loss: 0.1302
Epoch: 670, Loss: 0.1284
Epoch: 680, Loss: 0.1266
Epoch: 690, Loss: 0.1249
Epoch: 700, Loss: 0.1233
Epoch: 710, Loss: 0.1217
Epoch: 720, Loss: 0.1202
Epoch: 730, Loss: 0.1187
Epoch: 740, Loss: 0.1173
Epoch: 750, Loss: 0.1159
Epoch: 760, Loss: 0.1145
Epoch: 770, Loss: 0.1132
Epoch: 780, Loss: 0.1119
Epoch: 790, Loss: 0.1107
Epoch: 800, Loss: 0.1094
Epoch: 810, Loss: 0.1082
Epoch: 820, Loss: 0.1071
Epoch: 830, Loss: 0.1060
Epoch: 840, Loss: 0.1049
Epoch: 850, Loss: 0.1038
Epoch: 860, Loss: 0.1027
Epoch: 870, Loss: 0.1017
Epoch: 880, Loss: 0.1007
Epoch: 890, Loss: 0.0997
Epoch: 900, Loss: 0.0988
Epoch: 910, Loss: 0.0978
Epoch: 920, Loss: 0.0969
Epoch: 930, Loss: 0.0960
Epoch: 940, Loss: 0.0951
Epoch: 950, Loss: 0.0942
Epoch: 960, Loss: 0.0934
Epoch: 970, Loss: 0.0926
Epoch: 980, Loss: 0.0917
Epoch: 990, Loss: 0.0909
```

```
In [12]: sample = X_test[0]
         actual = y_test[0]
```

```
In [13]: def predict(X, w1, b1, w2, b2):
         Z1 = X @ w1 + b1
         A1 = relu(Z1)
         Z2 = A1 @ w2 + b2
         A2 = softmax(Z2)
         prediction = np.argmax(A2, axis=1)

         return prediction
```

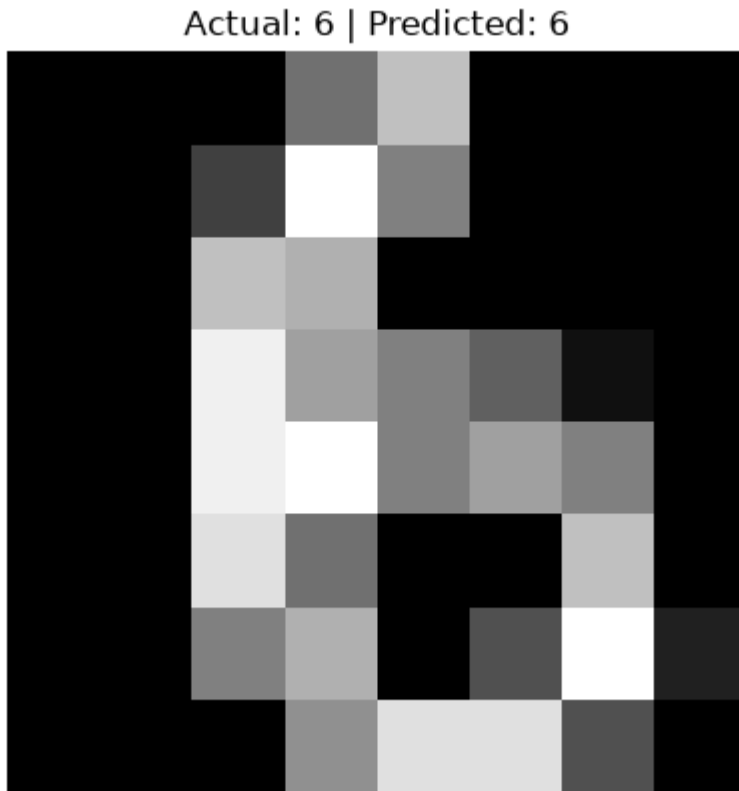
```
In [14]: pred = predict(
         X_test[0:1],
         w1, b1, w2, b2
         )
```

```
print("Actual:", actual)
print("Predicted:", pred[0])
```

Actual: 6

Predicted: 6

```
In [15]: plt.imshow(X_test[0].reshape(8, 8), cmap="gray")
plt.title(f"Actual: {y_test[0]} | Predicted: {pred[0]}")
plt.axis("off")
plt.show()
```



In []:

In []: